FACULDADE DE TECNOLOGIA DO ESTADO DE SÃO PAULO

DESENVOLVIMENTO DE SOFTWARE MULTIPLATAFORMA

ARTEFATOS DO PROJETO DE SOFTWARE

Teste de Desempenho Contínuo, Orientado por Rastreamento Ocular

Amanda de Oliveira Costa
Arthur Ribeiro Dias Fudali
Diego Baltazar de Souza Claudio
Giovana da Silva Albanês Santos
Igor Leite Gomes

Registro – SP
2026

Conteúdo

1 CANVAS: Modelo de Negócio e Estratégia **2**

2 SWOT: Análise Estratégica **3**
2.0.1 Fatores Internos: O Core do Projeto . . . 3
2.0.2 Fatores Externos: Oportunidades e Resiliência . . . 3
2.0.3 Síntese e Direcionamento . . . 4
2.1 Recomendações Estratégicas . . . 4

3 UX/UI: Design de Interface e Experiência do Usuário **5**
3.1 Protótipo . . . 5
3.2 Guia de Estilos . . . 5

4 Website: Estrutura e Especificações Técnicas Web **5**
4.1 Mapa do Site da Equipe . . . 5
4.2 Conteúdo das Páginas . . . 6
4.3 Especificações Técnicas do Site da Equipe . . . 6

5 Diagramas UML: Modelagem Visual do Sistema **6**
5.1 Diagrama de Casos de Uso . . . 6
5.2 Diagrama de Classes . . . 7
5.3 Diagrama de Objetos . . . 8

6 Banco de dados: Modelagem e Estruturação de Dados **9**

7 DevOps: Infraestrutura, Automação e Entrega Contínua **10**
7.1 Diagrama de Redes . . . 10
7.2 Pipeline CI/CD . . . 11
7.3 Configuração CI/CD . . . 12
7.4 Infraestrutura como Código . . . 15
7.5 Infraestrutura do Sistema . . . 16
7.5.1 Arquitetura de Rede . . . 16
7.5.2 Diagrama de Infraestrutura . . . 16
7.6 Arquitetura de Deploy . . . 18
7.7 Métricas e SLA . . . 18

1 CANVAS: Modelo de Negócio e Estratégia

O Business Model Canvas **?** é uma ferramenta visual estratégica[1] que permite descrever, analisar e projetar modelos de negócio de forma clara e objetiva. O canvas é dividido em nove blocos fundamentais que cobrem as principais áreas de um negócio.

Esse modelo de negócios foi estruturado de acordo com os nove blocos do Canvas, permitindo visualizar de forma integrada os principais elementos que compõem a proposta de valor, os recursos, os canais e a estrutura operacional do projeto FocusQuest.

Os Segmentos de Clientes alvo compreendem clínicas de neuropsicologia e psiquiatria, profissionais autônomos da saúde mental e instituições de reabilitação cognitiva. A Proposta de Valor consiste em desenvolver um serious game que avalia a atenção sustentada, impulsividade e tempo de resposta com base no Teste de Desempenho Contínuo (TDC), devolvendo um feedback imediato do resultado para cada usuário.

Os Canais de entrega envolvem o uso de computadores com câmeras e acesso à internet, com o sistema devidamente hospedado em servidor. O Relacionamento com o Cliente baseia-se em testes gamificados e interativos que permitem a geração de um pré-diagnóstico rápido por meio de um layout simples e intuitivo.

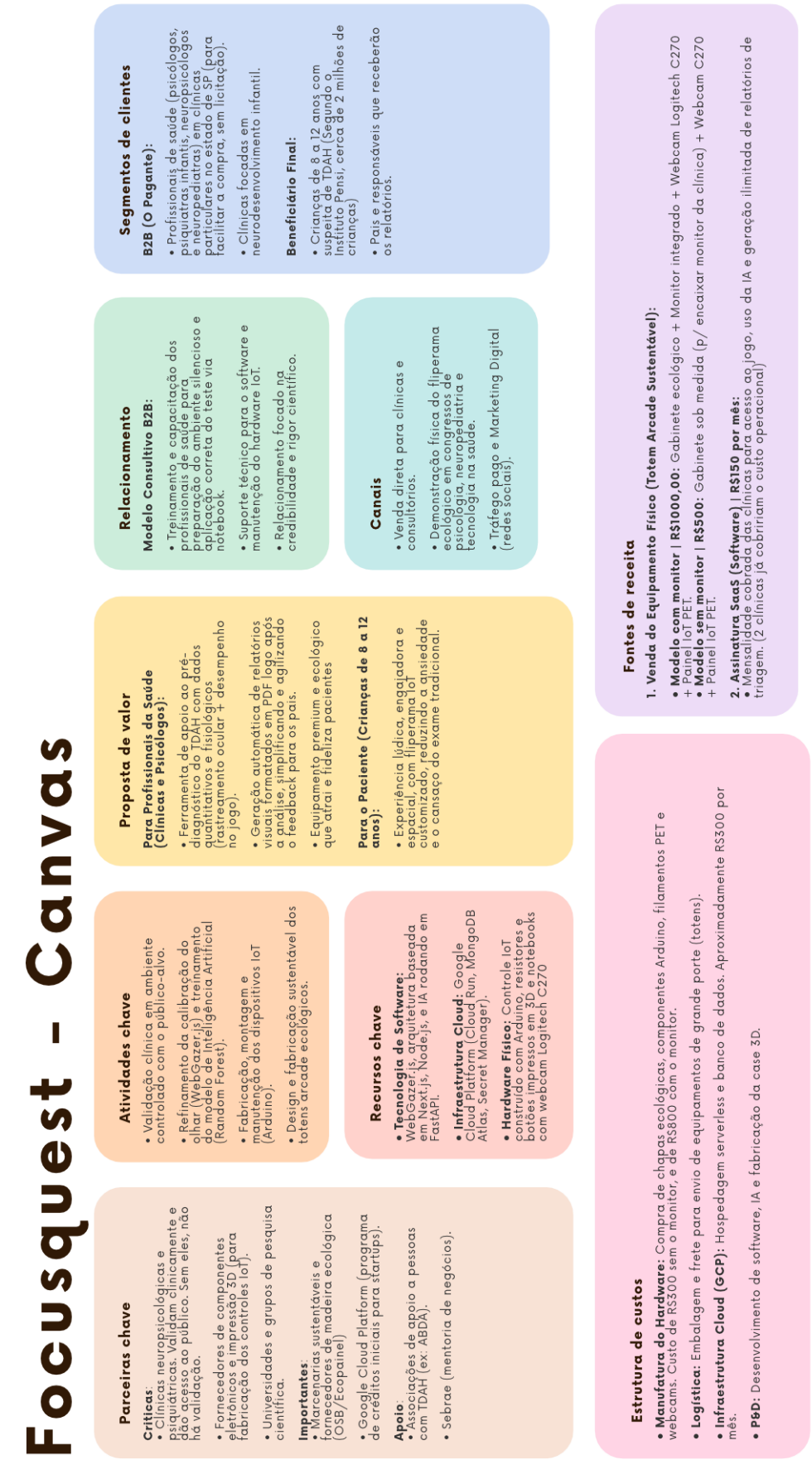
As Fontes de Renda são viabilizadas através de anúncios na internet, do tráfego no website, da disponibilidade do software para download e do sistema de recomendações de outros usuários. Entre os Recursos-Chave, destacam-se a facilidade de uso do software, a alta interatividade e a conveniência de gerar relatórios automatizados com interpretação de resultados.

As Atividades-Chave compreendem a criação de conteúdo educacional, o desenvolvimento e manutenção do sistema, além do suporte contínuo ao usuário. Os Parceiros-Chave foram identificados como as próprias clínicas de saúde mental e o usuário final, que colaboram com o ecossistema do projeto.

Por fim, a Estrutura de Custos é dividida entre custos fixos, como internet, hospedagem e investimento em tráfego pago, e custos variáveis, que abrangem o desenvolvimento de software, marketing, suporte e o patrocínio de profissionais em neuropsicologia e psiquiatria para validação do sistema.

[1] O Canvas foi desenvolvido por Alexander Osterwalder e é amplamente utilizado em startups e empresas estabelecidas para modelagem de negócios.

Figura 1: Business Model Canvas – PI - FocusQuest



2 SWOT: Análise Estratégica

A Análise SWOT (Strengths, Weaknesses, Opportunities, Threats) ou FOFA (Forças, Oportunidades, Fraquezas e Ameaças) **?** é uma ferramenta fundamental para planejamento estratégico[2] que permite identificar fatores internos e externos que impactam o projeto. Analisa forças, fraquezas, oportunidades e ameaças do projeto.

2.0.1 Fatores Internos: O Core do Projeto

No âmbito interno, o projeto destaca-se pelo uso de *eye-tracking* em um ambiente gamificado, o que transforma uma tarefa clínica geralmente exaustiva em uma experiência de alto engajamento, o que aumenta a fidelidade dos dados coletados e reduz a taxa de abandono do teste.

Por outro lado, a **Dependência de Hardware** é um ponto de atenção crítico. A eficácia da ferramenta está atrelada à qualidade da câmera e às condições de iluminação do usuário final. Isso exige um esforço contínuo em otimização de software e algoritmos de calibração para garantir que a solução seja acessível e confiável em diversos cenários domésticos ou clínicos.

2.0.2 Fatores Externos: Oportunidades e Resiliência

A crescente aceitação da triagem remota abre portas para parcerias B2B (*Business-to-Business*) com clínicas e instituições de ensino, posicionando o *FocusQuest* como uma solução de baixo custo e alta escalabilidade.

Entretanto, o projeto opera em um setor altamente regulado. A conformidade com a **LGPD (Lei Geral de Proteção de Dados)** e as normas de saúde representam ameaças estruturais que exigem atenção técnica e jurídica. Além disso, a possível entrada de grandes concorrentes exige que o projeto mantenha agilidade na inovação e foco na experiência do usuário para preservar sua diferenciação de mercado.

2.0.3 Síntese e Direcionamento

Em suma, a SWOT indica que o sucesso do *FocusQuest* depende da capacidade de converter sua superioridade técnica e científica em uma ferramenta de fácil adoção. O foco estratégico imediato reside em reduzir as barreiras técnicas de entrada enquanto se fortalecem as barreiras de saída através do rigor científico e da fidelização dos profissionais da área.

[2] A análise SWOT foi criada na década de 1960 por Albert Humphrey no Stanford Research Institute.

Tabela 1: Análise SWOT Estratégica - Projeto FocusQuest

FATORES INTERNOS (Contornáveis)	
FORÇAS (Strengths)	**FRAQUEZAS (Weaknesses)**
• **Inovação:** Diferencial competitivo com uso de *eye-tracking* acessível. • **Engajamento:** Metodologia de *Serious Game* que reduz o abandono em testes clínicos. • **Rigor Científico:** Alinhamento técnico com o protocolo TDC.	• **Dependência de Hardware:** Sensibilidade a variações de iluminação e qualidade de câmeras externas. • **Barreiras de Dados:** Complexidade na aquisição de bases de dados clínicas robustas para validação. • **Calibração:** Necessidade de ajuste inicial que pode afetar a experiência do usuário.
FATORES EXTERNOS (Incontroláveis)	
OPORTUNIDADES (Opportunities)	**AMEAÇAS (Threats)**
• **Expansão Digital:** Crescente demanda por ferramentas de triagem remota. • **Parcerias B2B:** Potencial de adoção por instituições de ensino e clínicas neuropsicológicas. • **Diferenciação de Mercado:** Inexistência de soluções gamificadas semelhantes no cenário nacional.	• **Compliance e LGPD:** Rigor na proteção de dados sensíveis e diagnósticos de saúde. • **Barreiras de Adoção:** Resistência de profissionais tradicionais a métodos de triagem automatizados. • **Concorrência:** Entrada de grandes concorrentes com maior capacidade de investimento.

2.1 Recomendações Estratégicas

Com base no cruzamento das variáveis da análise SWOT, as diretrizes estratégicas para o desenvolvimento do *FocusQuest* devem focar em quatro eixos de atuação:

- **Estratégias Ofensivas (Forças + Oportunidades):** Explorar o eye-tracking para posicionar o produto como líder no nicho de triagem remota, utilizando a gamificação como diferencial de mercado frente a métodos tradicionais.
- **Estratégias de Confronto (Forças + Ameaças):** Utilizar o **Rigor Científico** (alinhamento ao TDC) e os dados de engajamento para mitigar a resistência de profissionais tradicionais, estabelecendo o sistema como uma ferramenta de apoio diagnóstico confiável e difícil de ser replicada por concorrentes genéricos.
- **Estratégias de Reforço (Fraquezas + Oportunidades):** Firmar parcerias estratégicas **B2B** com clínicas e instituições de ensino para facilitar a coleta de bases de dados clínicas, superando a barreira de validação e acelerando o aprimoramento dos algoritmos de rastreamento.
- **Estratégias Defensivas (Fraquezas + Ameaças):** Estabelecer protocolos rígidos de **Compliance e LGPD** para proteção de dados sensíveis e investir em UX (*User Experience*) para simplificar a calibração de hardware, evitando que limitações técnicas ou burocráticas abram espaço para a concorrência.

3 UX/UI: Design de Interface e Experiência do Usuário

UX (User Experience) e UI (User Interface) **?** são disciplinas complementares focadas em criar produtos digitais que sejam funcionais, acessíveis e agradáveis de usar. O UX concentra-se na experiência geral do usuário, enquanto o UI cuida dos elementos visuais e interativos.

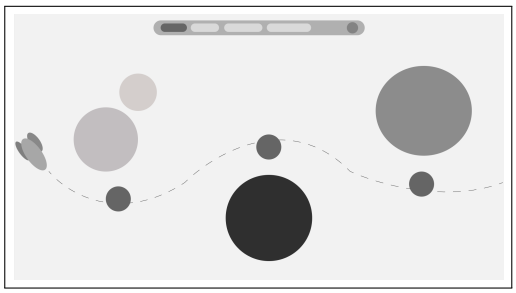


Figura 2: Wireframe da tela inicial

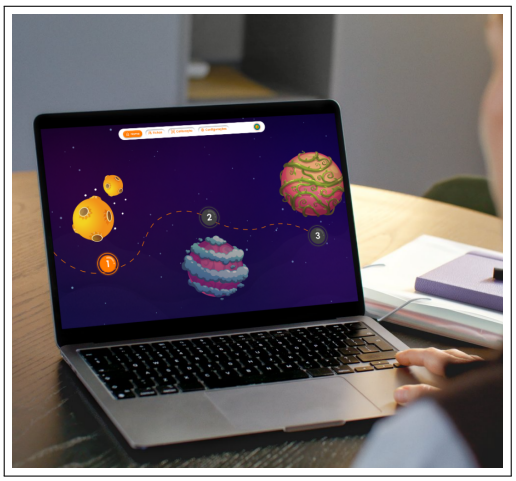


Figura 3: Mockup de alta fidelidade

3.1 Protótipo

O protótipo do projeto foi desenvolvido utilizando a ferramenta Figma, ferramenta de design poderosa e colaborativa para equipes.

3.2 Guia de Estilos

Tabela 2: Paleta de Cores do Projeto

Elemento	Cor	Código HEX	Uso
Primária		#FF7B00	Botões principais, destaques, Títulos
Texto		#3B3B3B	Texto principal
Fundo		#FFFFFF	Background padrão
Sucesso		#00c950	Mensagens de sucesso
Erro		#fb2c36	Mensagens de erro

4 Website: Estrutura e Especificações Técnicas Web

4.1 Mapa do Site da Equipe

O mapa do site (sitemap) mostra a hierarquia e organização das páginas do website da equipe:

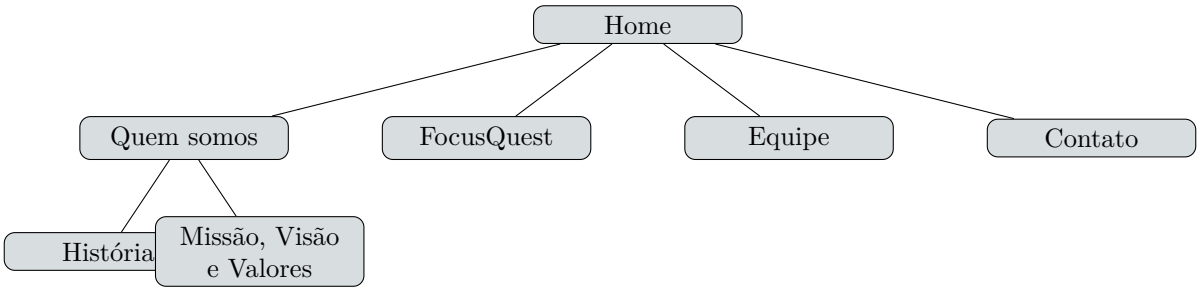


Figura 4: Estrutura de navegação do website da equipe

4.2 Conteúdo das Páginas

Páginas principais do site da equipe:

- **Home:** Apresentação da equipe e logo
- **Quem somos:** História da equipe, missão, visão e valores
- **FocusQuest:** Descrição do projeto, objetivos e mockup
- **Equipe:** Perfil dos membros (nome, foto, função, LinkedIn/GitHub/Instagram)
- **Contato:** Formulário de contato e e-mail

4.3 Especificações Técnicas do Site da Equipe

Tabela 3: Stack Tecnológica do Website da Equipe

Camada	Tecnologia	Descrição
Frontend	JavaScript	Linguagem de programação para desenvolvimento web
Estilização	CSS	Linguagem de estilo para definição da aparência dos elementos HTML
Hospedagem	Vercel	Plataforma de deploy e hosting gratuita
Formulário	EmailJS	Biblioteca para envio de mensagens de contato via e-mail
Analytics	Google Analytics	Ferramenta de métricas e análise de visitantes

5 Diagramas UML: Modelagem Visual do Sistema

A Unified Modeling Language (UML) **?** é uma linguagem de modelagem visual usada para especificar, visualizar, construir e documentar artefatos de sistemas de software.[3]

5.1 Diagrama de Casos de Uso

O diagrama de casos de uso do *FocusQuest* descreve as interações entre os atores principais (**Profissional de Saúde**, **Paciente** e **Administrador**) e os componentes do sistema. O processo central consiste na "Sessão de Teste", coordenada pelo profissional, que integra a calibração ocular e as três fases de avaliação do desempenho atencional. A operação é suportada tecnicamente por *WebGazer*, *Socket.IO* e *MongoDB*, culminando na geração automática de um relatório de desempenho para auxílio no pré-diagnóstico.

[3] A UML foi padronizada pela Object Management Group (OMG) em 1997.

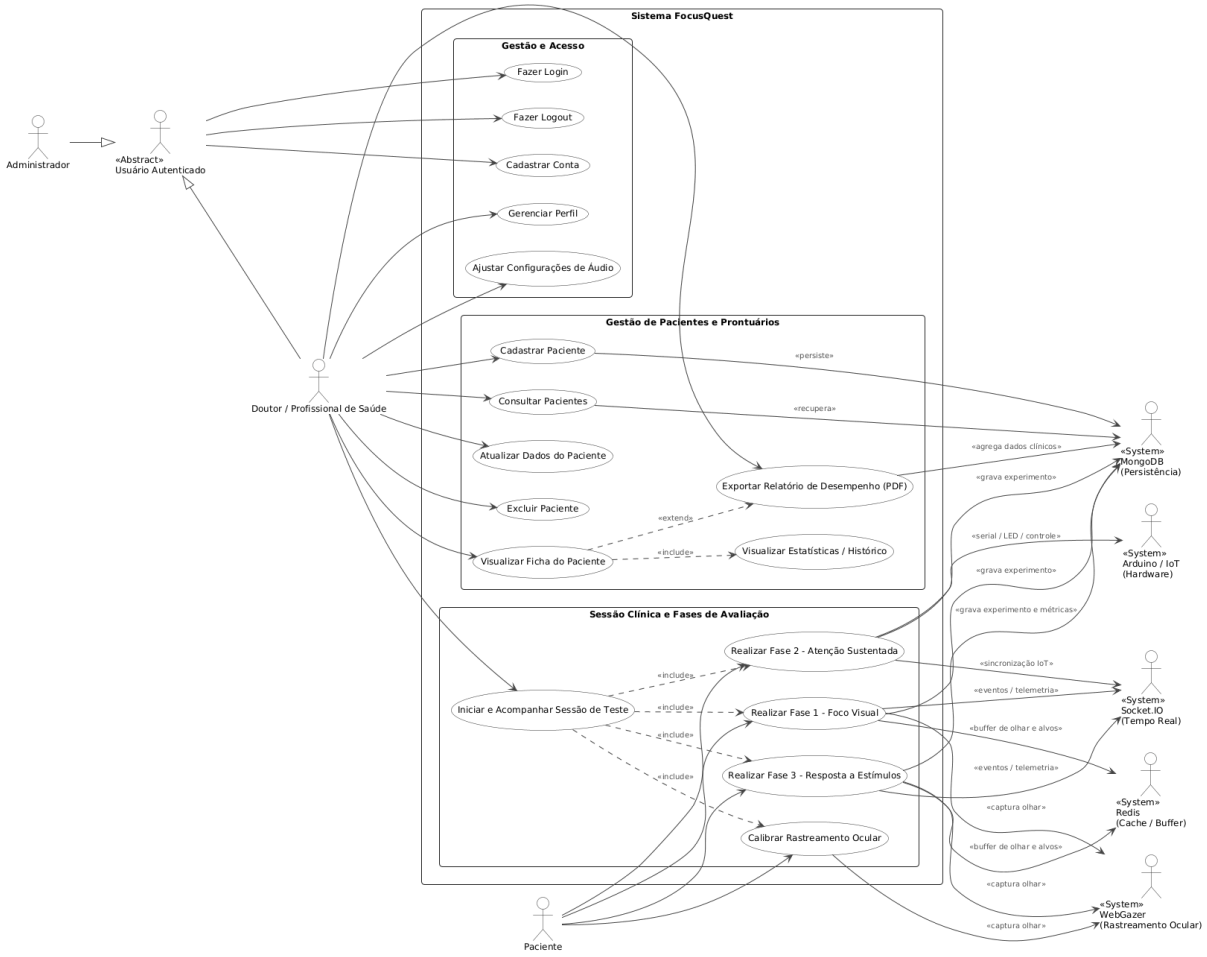


Figura 5: Diagrama de Casos de Uso do Sistema

5.2 Diagrama de Classes

O diagrama de classes do sistema *FocusQuest* detalha a arquitetura estrutural e a lógica de persistência do *backend*. A solução é estruturada em torno de um componente **Server** que orquestra diferentes **Handlers** e **Services** (Fases 1, 2 e 3), responsáveis por processar as regras de negócio e a interação em tempo real.

As entidades de dados fundamentais são o **Usuário** (profissional) e o **Paciente**, com o sistema a gerir a relação de acompanhamento e a criação de **Experimentos** específicos para cada fase do teste. Estes experimentos registam métricas detalhadas, como o histórico de rastreio ocular e tempos de resposta, que são posteriormente processados pelas classes de **Estatísticas** para gerar resumos analíticos. A infraestrutura é completada por módulos de **DatabaseConnections** para persistência em MongoDB e Redis, e uma integração com **Arduino** para comunicação com dispositivos de *hardware* externos.

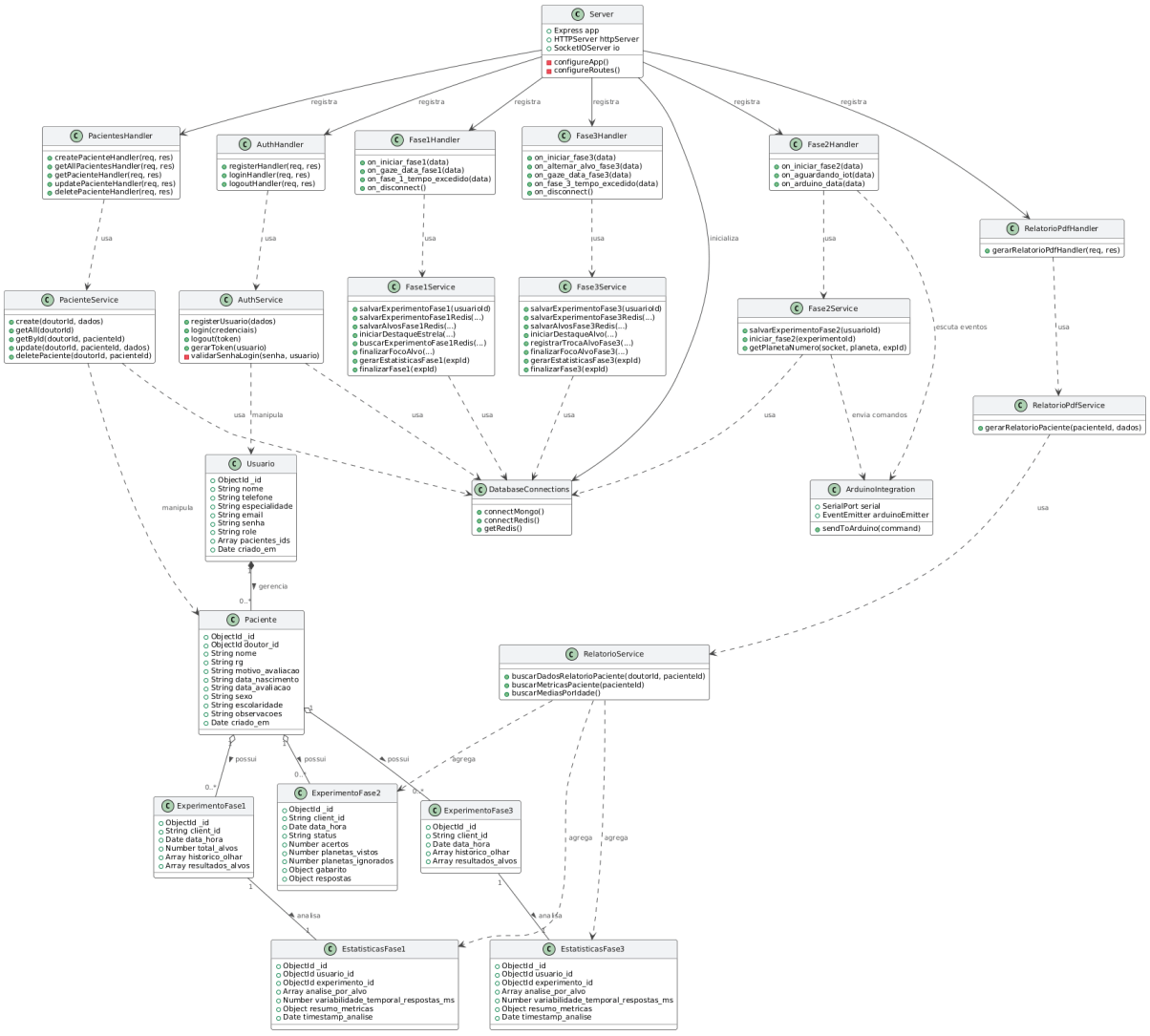


Figura 6: Diagrama de Classes do Sistema

5.3 Diagrama de Objetos

O diagrama de objetos apresenta um retrato do sistema *FocusQuest* em tempo de execução durante uma sessão clínica, validando o fluxo de dados e a arquitetura na prática. A representação ilustra as instâncias de infraestrutura ativas (servidor e banco de dados) interligadas aos atores do processo, onde a profissional (usuário) gerencia a avaliação de um paciente. Ao longo da sessão, o sistema instancia as três fases do experimento, registrando dados brutos como coordenadas de rastreamento ocular e interações. Em seguida, esses registros são processados automaticamente em instâncias de Estatísticas, que consolidam métricas precisas, como tempo médio de reação e total de acertos, retornando um relatório analítico pronto para a visualização da profissional e auxiliando diretamente no pré-diagnóstico.

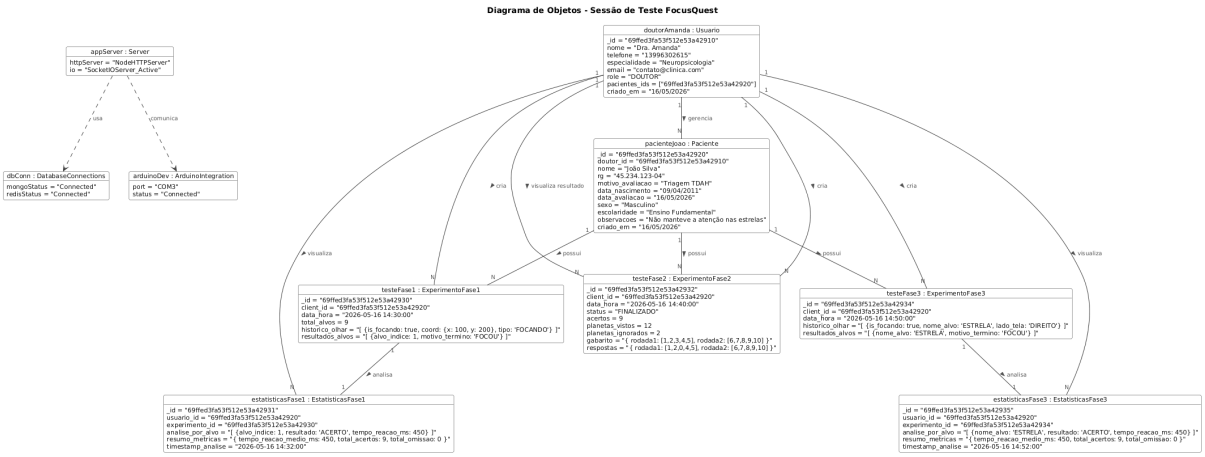


Figura 7: Diagrama de Objetos do Sistema

6 Banco de dados: Modelagem e Estruturação de Dados

O modelo de dados do MongoDB para o *FocusQuest* adota uma abordagem NoSQL orientada a documentos, otimizada para o armazenamento de dados biométricos e clínicos com alta flexibilidade. O esquema organiza-se em coleções fundamentais: **Usuários**, que gere o acesso e perfil dos profissionais de saúde; **Pacientes**, que centraliza o histórico e prontuário dos avaliados; e as coleções de **Experimentos** e **Estatísticas**, responsáveis pela persistência detalhada de cada fase do teste. A modelagem utiliza o conceito de referenciamento para manter a integridade entre profissionais e seus respectivos pacientes, permitindo que o sistema processe e recupere padrões complexos de rastreamento ocular e tempos de reação de forma eficiente para a composição dos relatórios finais.

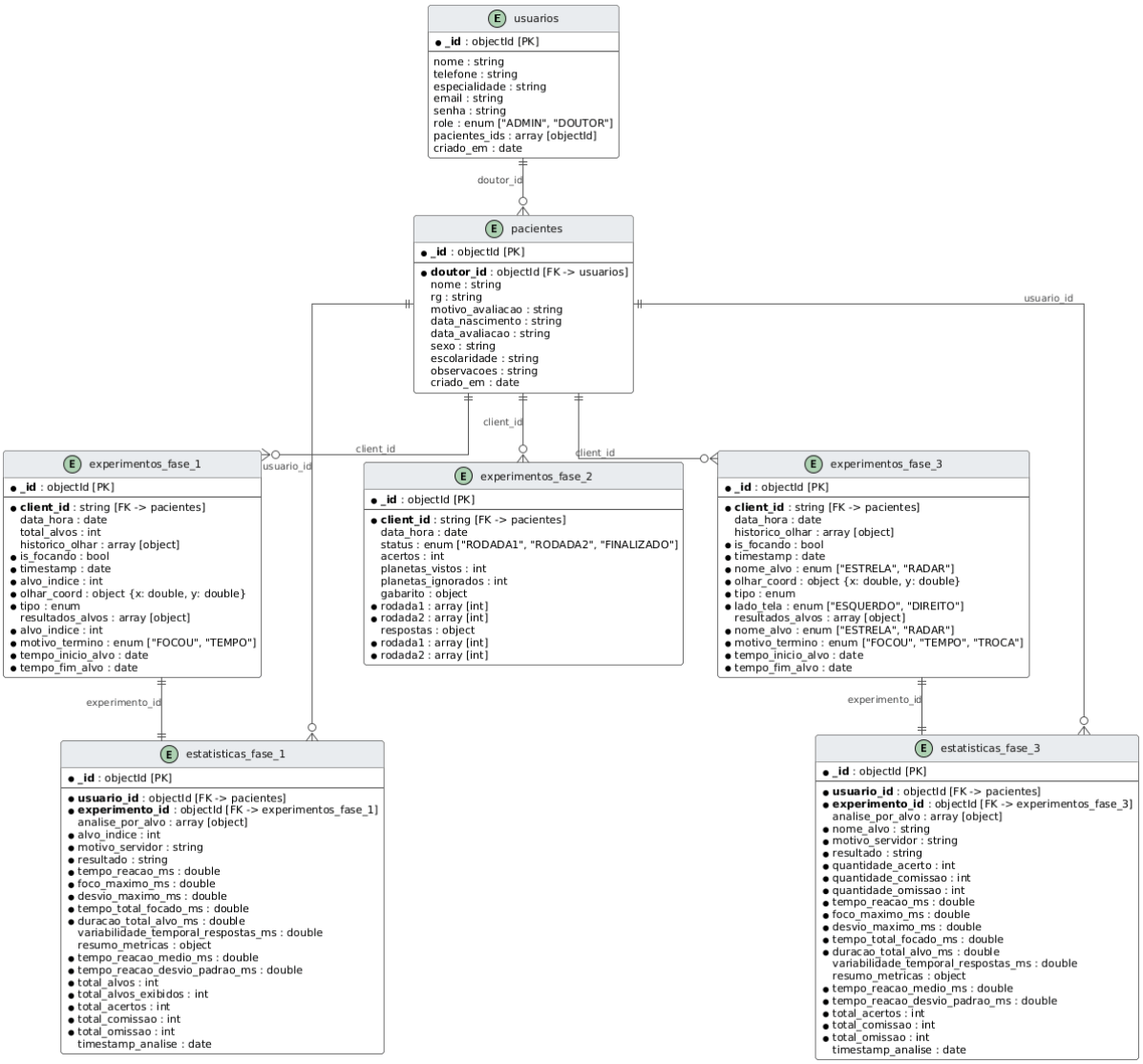


Figura 8: Diagrama de Banco de Dados do Sistema

7 DevOps: Infraestrutura, Automação e Entrega Contínua

DevOps **?** é uma cultura e conjunto de práticas que integra desenvolvimento de software (Dev) e operações de TI (Ops), focando em automação, colaboração e entrega contínua.

7.1 Diagrama de Redes

O diagrama de redes ilustra a arquitetura de infraestrutura e o fluxo de comunicação do sistema *FocusQuest*, projetado para suportar o TDC orientado por visão computacional. O fluxo tem início no PC do usuário, onde ocorre a captura local dos dados de rastreamento ocular (*eye-tracking*), que são transmitidos de forma segura através do roteador e do provedor de acesso à internet (ISP). Ao ingressar no ambiente de *Virtual Private Cloud* (VPC), o tráfego é gerenciado e distribuído por um *Cloud Load Balancing*, garantindo estabilidade antes de atingir a API responsável pela coleta de dados. Para assegurar alto desempenho e confiabilidade, a arquitetura de armazenamento combina o *Memorystore* (Redis) para operações de cache em tempo real com um banco de dados *Cloud NoSQL* (MongoDB) para a persistência definitiva. Complementando o ecossistema, os registros consolidados são enviados para um módulo de Inteligência Artificial

de aprendizagem guiada, responsável por processar os dados biométricos e gerar os resultados preditivos sobre a atenção e o foco do paciente.

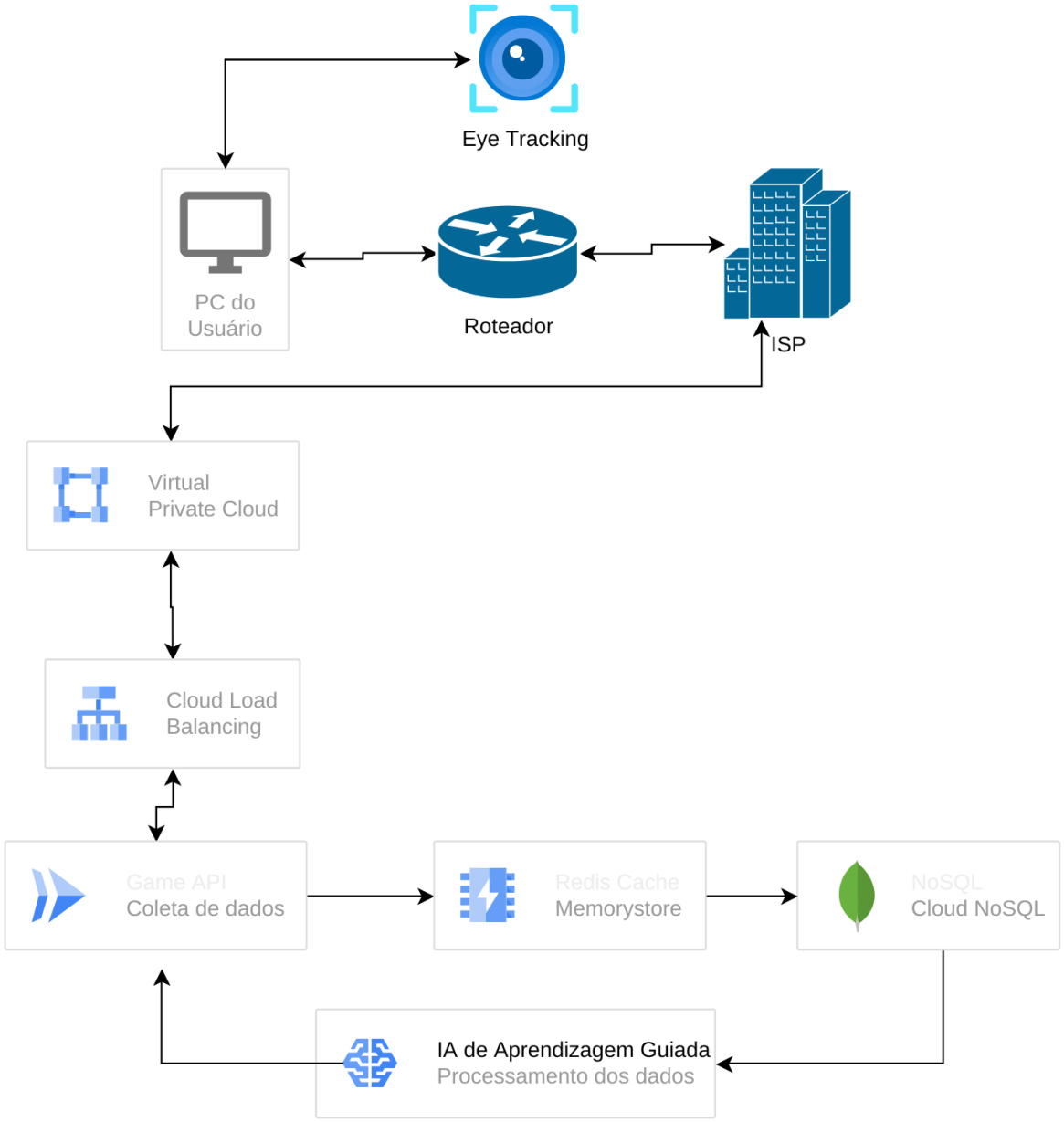


Figura 9: Diagrama de Redes do Sistema

7.2 Pipeline CI/CD

O pipeline de integração e entrega contínua do projeto é definido em `cloudbuild.yaml`. O fluxo automatiza a construção das imagens Docker, o envio para o Artifact Registry e o deploy dos serviços frontend e backend no Cloud Run. O backend é publicado com parâmetros de execução específicos, como `NODE extunderscore ENV=production`, `PORT=4000`, `min-instances=3` e `max-instances=20`. O frontend é publicado separadamente, com `PORT=3000`, escalabilidade menor e variável `NEXT extunderscore PUBLIC extunderscore API extunderscore URL` injetada via Secret Manager.

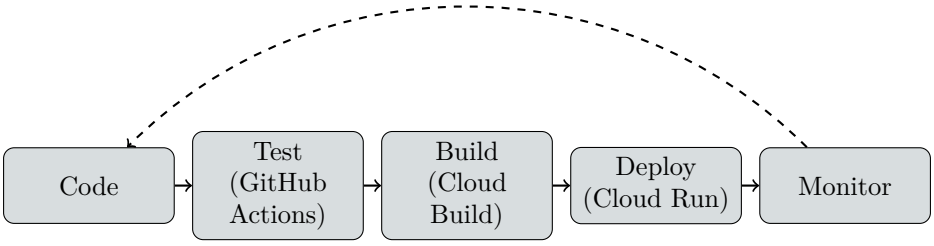


Figura 10: Pipeline CI/CD do FocusQuest

7.3 Configuração CI/CD

```
steps:

# Backend - Build Docker image

- name: 'gcr.io/cloud-builders/docker'
id: 'backend-build'
args:
- 'build'
- '-t'
- 'us-central1-docker.pkg.dev/$PROJECT_ID/focusquest/backend:$SHORT_SHA'
- '-t'
- 'us-central1-docker.pkg.dev/$PROJECT_ID/focusquest/backend:latest'
- '-f'
- 'backend-rastreamento-ocular/Dockerfile'
- 'backend-rastreamento-ocular/'
waitFor: ['-']



# Backend - Push to Artifact Registry

- name: 'gcr.io/cloud-builders/docker'
id: 'backend-push'
args:
- 'push'
- 'us-central1-docker.pkg.dev/$PROJECT_ID/focusquest/backend:$SHORT_SHA'
waitFor: ['backend-build']



# Frontend - Build Docker image

- name: 'gcr.io/cloud-builders/docker'
id: 'frontend-build'
args:
- 'build'
- '-t'
- 'us-central1-docker.pkg.dev/$PROJECT_ID/focusquest/frontend:$SHORT_SHA'
```

```
- '-t'
- 'us-central1-docker.pkg.dev/$PROJECT_ID/focusquest/frontend:latest'
- '-f'
- 'FocusQuest-web/Dockerfile'
- 'FocusQuest-web/'
waitFor: ['-']



# Frontend - Push to Artifact Registry

- name: 'gcr.io/cloud-builders/docker'
id: 'frontend-push'
args:
- 'push'
- 'us-central1-docker.pkg.dev/$PROJECT_ID/focusquest/frontend:$SHORT_SHA'
waitFor: ['frontend-build']



# Backend - Deploy to Cloud Run

- name: 'gcr.io/cloud-builders/run'
id: 'backend-deploy'
args:
- 'deploy'
- 'focusquest-backend'
- '--image'
- 'us-central1-docker.pkg.dev/$PROJECT_ID/focusquest/backend:$SHORT_SHA'
- '--region'
- 'us-central1'
- '--platform'
- 'managed'
- '--memory'
- '1Gi'
- '--cpu'
- '2'
- '--vpc-connector'
- 'focusquest-connector'
- '--vpc-egress'
- 'private-ranges-only'
- '--timeout'
- '3600'
- '--min-instances'
- '3'
- '--max-instances'
- '20'
- '--allow-unauthenticated'
- '--set-env-vars'
- 'NODE_ENV=production,PORT=4000,REDIS_PORT=6379,JWT_EXPIRACAO=1d'
- '--update-secrets'
```

```
- 'MONGO_URI=MONGO_URI:latest,REDIS_HOSTNAME=REDIS_HOSTNAME:latest,JWT_SECRET=JWT_SECRET:lat
env:
- 'CLOUDSDK_RUN_REGION=us-central1'
waitFor: ['backend-push']



# Frontend - Deploy to Cloud Run

- name: 'gcr.io/cloud-builders/run'
id: 'frontend-deploy'
args:
- 'deploy'
- 'focusquest-frontend'
- '--image'
- 'us-central1-docker.pkg.dev/$PROJECT_ID/focusquest/frontend:$SHORT_SHA'
- '--region'
- 'us-central1'
- '--platform'
- 'managed'
- '--memory'
- '512Mi'
- '--cpu'
- '1'
- '--timeout'
- '3600'
- '--min-instances'
- '2'
- '--max-instances'
- '10'
- '--allow-unauthenticated'
- '--set-env-vars'
- 'NODE_ENV=production,PORT=3000'
- '--update-secrets'
- 'NEXT_PUBLIC_API_URL=API_URL:latest'
env:
- 'CLOUDSDK_RUN_REGION=us-central1'
waitFor: ['frontend-push']



# Configuração de imagens

images:

- 'us-central1-docker.pkg.dev/$PROJECT_ID/focusquest/backend:$SHORT_SHA'
- 'us-central1-docker.pkg.dev/$PROJECT_ID/focusquest/backend:latest'
- 'us-central1-docker.pkg.dev/$PROJECT_ID/focusquest/frontend:$SHORT_SHA'
- 'us-central1-docker.pkg.dev/$PROJECT_ID/focusquest/frontend:latest'

# Opções de build
```

```
options:
machineType: 'N1_HIGHCPU_8'
logging: CLOUD_LOGGING_ONLY
substitution_option: 'ALLOW_LOOSE'

# Timeout total

timeout: '3600s'

# Substituições customizadas

substitutions:
_REGION: 'us-central1'
_SERVICE_ACCOUNT: 'focusquest-deploy'
```

7.4 Infraestrutura como Código

Tabela 4: Ferramentas DevOps Utilizadas

Categoria	Ferramenta	Finalidade
Controle de Versão	Git + GitHub	• Versionamento do código-fonte e acionamento de pipelines
Execução de aplicações	Cloud Run	• Hospedagem serverless de frontend e backend com auto scaling e HTTPS
Containerização	Docker	• Empacotamento do frontend e backend
Balanceamento de carga	Cloud Load Balancer	• Distribuição de tráfego, TLS/SSL e suporte a WebSocket
CDN	Cloud CDN	• Cache global para assets estáticos (imagens, CSS, JS)
Cache / Mensageria	Memorystore for Redis	• Cache em tempo real e comunicação distribuída entre instâncias
Banco de dados	MongoDB Atlas	• Persistência principal dos dados da aplicação
Armazenamento	Cloud Storage	• Armazenamento de backups e logs com versionamento e recuperação
Gerenciamento de segredos	Secret Manager	• Armazenamento seguro de credenciais e variáveis sensíveis
Monitoramento	Cloud Monitoring e Logging	• Observabilidade de CPU, memória, latência e centralização de logs
CI/CD	Cloud Build	• Automação de build, push de imagens e deploy contínuo
Rede	VPC Network	• Isolamento de rede e comunicação privada entre serviços
Provisionamento	Scripts shell	• Setup da rede, Redis, Artifact Registry e deploy

7.5 Infraestrutura do Sistema

Esta seção documenta como o sistema FocusQuest funciona em diferentes ambientes e plataformas.

7.5.1 Arquitetura de Rede

Tabela 5: Componentes de Infraestrutura

Componente	Descrição
Ambiente Local (Desenvolvimento)	• Backend Node.js e frontend Next.js executados localmente para desenvolvimento • Banco MongoDB e cache Redis via Docker Compose para testes e validações • Configuração por variáveis de ambiente para simular integrações de produção • Suporte a testes manuais e automatizados antes do deploy
Cloud (Produção/Staging)	• Serviços frontend e backend implantados em Cloud Run • Build, push de imagens e deploy automatizados com Cloud Build • Artifact Registry para versionamento e armazenamento das imagens Docker • Auto-scaling configurado por serviço (frontend e backend com limites próprios) • Integração com VPC Connector para acesso privado a serviços internos
Dados e Mensageria	• MongoDB para persistência de usuários, pacientes, experimentos e resultados • Redis para cache, sessão e sincronização em tempo real • Suporte a comunicação distribuída do Socket.io em múltiplas instâncias • Estratégia de retenção e backup para continuidade operacional
Cliente e Tempo Real	• Aplicação web com rastreamento ocular híbrido (processamento local + backend) • Comunicação REST para operações de negócio e WebSocket para eventos em tempo real • Encaminhamento de rotas de API do frontend para backend por configuração de ambiente • Atualização de análises em tempo real para fases experimentais

7.5.2 Diagrama de Infraestrutura

Para ilustrar como os componentes se conectam:

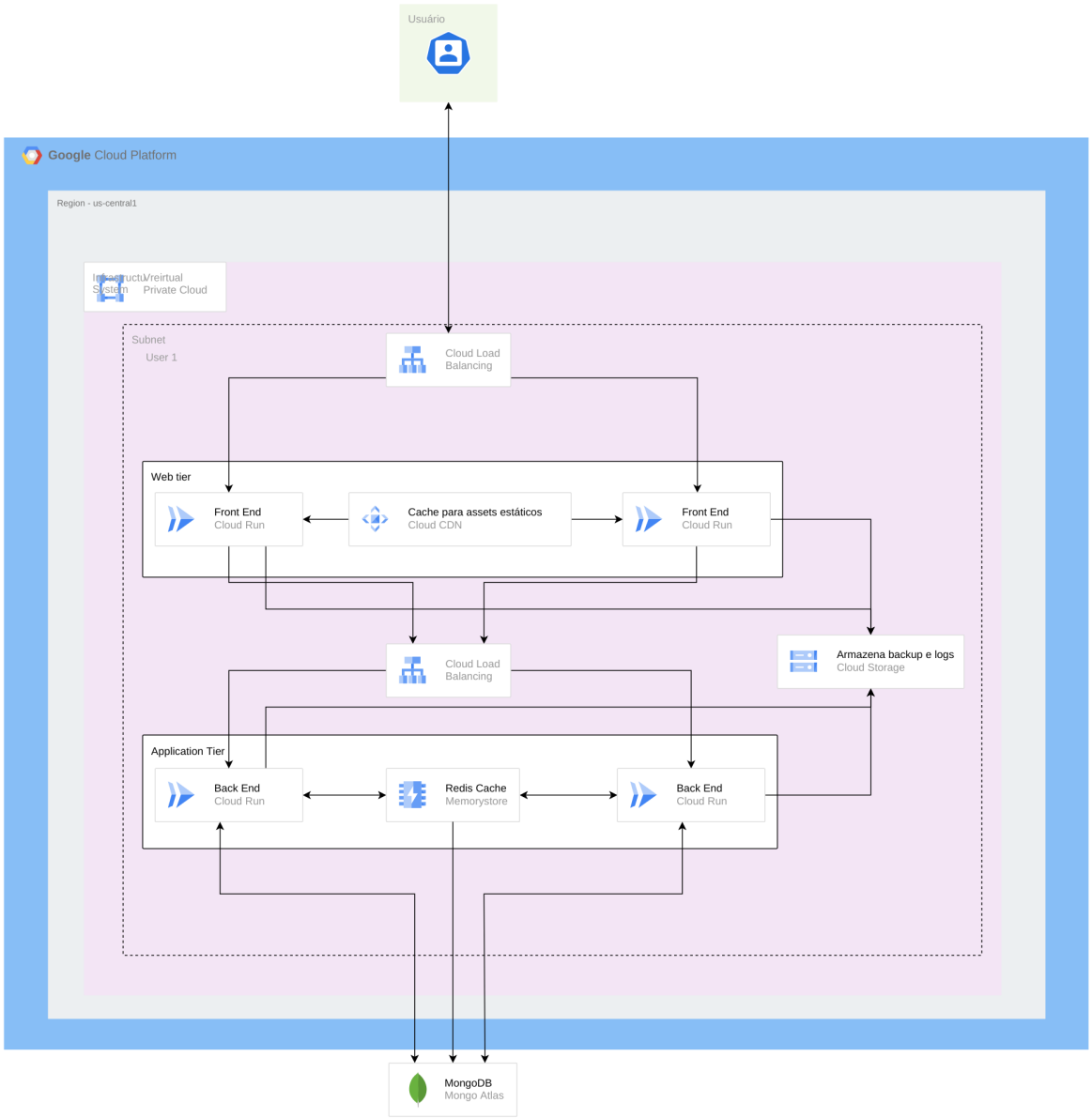


Figura 11: Diagrama de Infraestrutura do Sistema

Exemplo de descrição:

- **Usuários:** Acessam a aplicação direcionados primeiramente por um balanceador de carga externo.
- **Balanceamento de Carga:** Cloud Load Balancing é utilizado em duas camadas (tráfego externo para a camada Web e tráfego interno da camada Web para a camada de Aplicação).
- **Web Tier (Front End):** Instâncias executadas em contêineres utilizando o Cloud Run.
- **Application Tier (Back End):** Serviços de retaguarda e API também executados de forma serverless através do Cloud Run.
- **Cache Estático (CDN):** Cloud CDN utilizado na camada Web para entrega rápida e otimizada de assets estáticos.

- **Cache de Aplicação:** Redis utilizado na camada de Back End para otimização de performance e redução de latência.
- **Banco de Dados:** MongoDB (hospedado de forma externa no Mongo Atlas) comunicando-se diretamente com o Back End.
- **Storage / Backups e Logs:** Cloud Storage utilizado de forma centralizada para armazenar os logs e backups tanto do Front End quanto do Back End.
- **Rede / Infraestrutura:** Ambiente contido dentro do Google Cloud Platform (GCP), isolado em uma VPC e sub-rede na região us-central1.

7.6 Arquitetura de Deploy

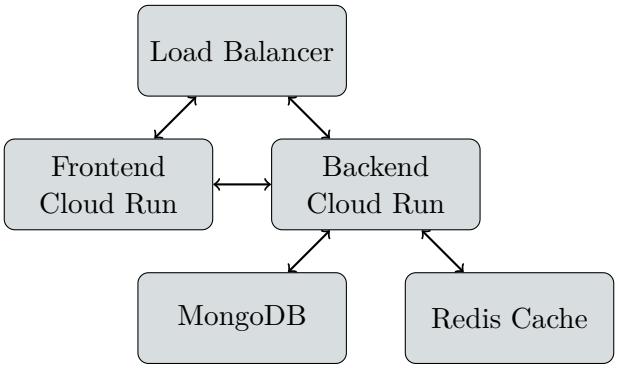


Figura 12: Arquitetura de Deployment

7.7 Métricas e SLA

Para o ambiente do FocusQuest, as métricas de operação podem ser definidas com base na arquitetura em Cloud Run, com backend Node.js, frontend Next.js, Redis e MongoDB.

- **Disponibilidade:** 99.5% a 99.9%
- **Tempo de Resposta Médio:** $<$ 300ms para rotas REST simples
- **Tempo de Resposta P95:** $<$ 800ms
- **Taxa de Erro:** $<$ 1%
- **Throughput:** 300 a 800 requisições/segundo em carga normal
- **Latência WebSocket:** $<$ 100ms para eventos em tempo real
- **Deploy Frequency:** diário ou sob demanda
- **Lead Time:** $<$ 2 horas para alterações pequenas
- **MTTR:** $<$ 30 minutos
- **Escalabilidade Backend:** 3 a 20 instâncias
- **Escalabilidade Frontend:** 2 a 10 instâncias

Conclusão

Este documento apresenta os principais artefatos necessários para a documentação completa de um projeto de software. Cada seção fornece exemplos práticos de como inserir e formatar os diferentes tipos de artefatos utilizando LaTeX.

Os artefatos documentados incluem:

- **CANVAS:** Modelo de negócio e estratégia
- **SWOT:** Análise estratégica de forças, fraquezas, oportunidades e ameaças
- **UX/UI:** Design de interface e experiência do usuário
- **Website:** Estrutura e especificações técnicas web
- **Diagramas UML:** Modelagem visual do sistema
- **Banco de Dados:** Modelagem e estruturação de dados
- **DevOps:** Infraestrutura, automação e práticas de entrega contínua